

Projekt: System do zarządzania szkołą korepetycji

Nazwa kodowa: SCUOLA

Skład zespołu:

Bartosz Badura (Lider)
Eryk Witkowski

Linki interaktywne:

[Repozytorium GitHub](#)

[Tablica trello](#)

9 maja 2026

Wprowadzenie

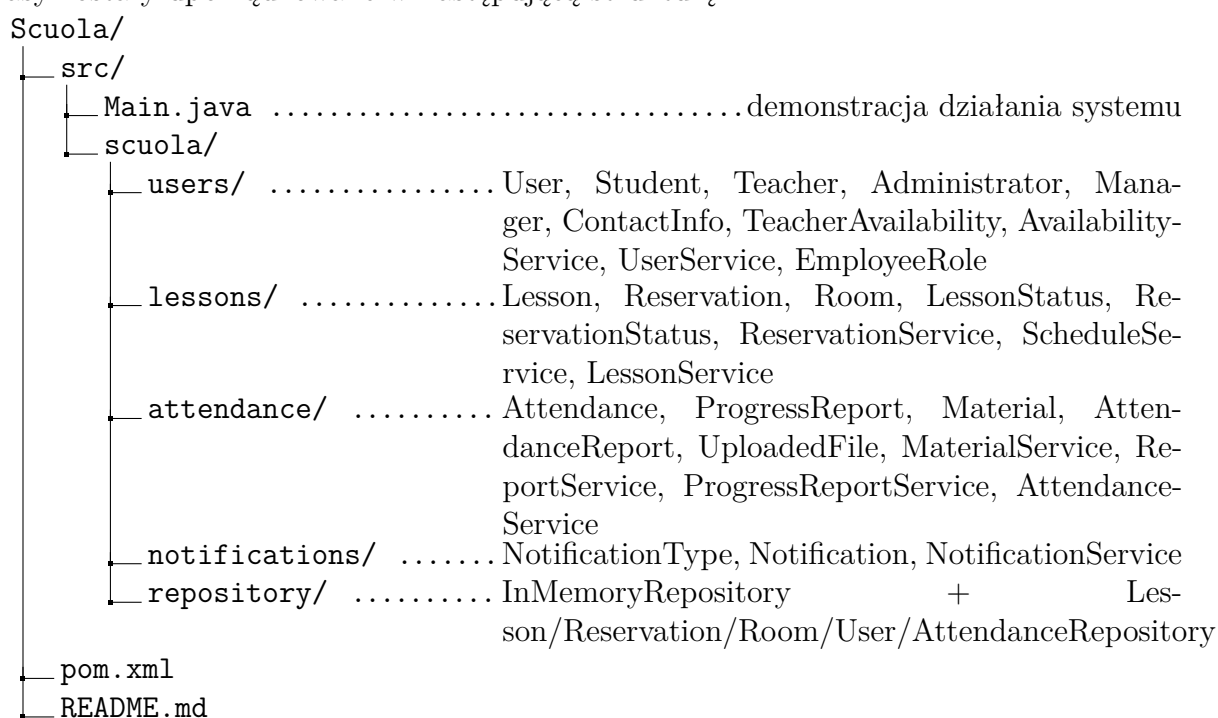
Poniższa sekcja zawiera opis wykonanych zadań w ostatnim sprincie

1 Implementacja back-endu

Przez narzut czasowi i dla uproszczenia pracy, implementacja została zrobiona w javie, bez implementacji zamierzonej wersji webowej. Można ją traktować jako proof of concept pokazujący zamierzone działanie systemu. Implementacja opierała się na diagramach UML i sekwencji opracowanych w poprzednim sprincie.

1.1 Klasy

Klasy zostały uporządkowane w następującą strukturę:



bardziej szczegółowy opis poszczególnych funkcjonalności znajduje się w raporcie 3. (diagramy UML) i README.md

1.2 Testy

Dla sprawdzenia poprawności napisanych klas napisaliśmy łącznie 164 testy, w tym:

- 151 testy jednostkowe, opisujące każdą z klas
- 17 testy akceptacyjne, opisujące scenariusze w formacie GWT z raportu 2

1.2.1 Testy jednostkowe

W ramach testów jednostkowych zdecydowaliśmy się sprawdzić:

- attendance

- AttendanceReport - czy pusty raport zwraca współczynnik obecności 0, czy rekordy są read-only, czy poprawnie oblicza współczynnik obecności i czy poprawnie zwraca liczbę obecnych, nieobecnych i objętych raportem
 - AttendanceService - czy oznaczenie obecności zapisuje wpis na lekcji i u ucznia, czy odrzuca argumenty null, czy aktualizacja zmienia status obecności, czy zwraca listę uczniów zapisanych na zajęcia, czy próg nieobecności da się skonfigurować i czy poprawnie wykrywa jego przekroczenie
 - Attendance - czy konstruktor zapisuje wszystkie pola (data, obecność, uczeń, lekcja) i czy status obecności da się przełączyć
 - Material - czy konstruktor zapisuje wszystkie pola i czy skrócony konstruktor ustawia datę przesłania
 - ProgressReport - czy konstruktor zapisuje wszystkie pola i czy treść notatki da się zaktualizować
 - UploadedFile - czy konstruktor zapisuje pola, czy skrócony konstruktor ustawia pustą zawartość i czy settery aktualizują pola
 - MaterialService - czy poprawny plik zostaje dodany, czy odrzuca plik za duży (limit 100 MB), czy odrzuca rozmiar niedodatni, czy odrzuca argumenty null, czy usuwa materiał o pasującej nazwie, czy zwraca false gdy materiału brak, czy zwraca pustą listę dla null lekcji, czy pobranie zwraca plik o tej samej nazwie i czy zwraca null dla null materiału
 - ProgressReportService - czy utworzenie łączy notatkę z uczniem i autorem, czy odrzuca pustą treść, czy odrzuca brak ucznia lub lekcji, czy aktualizacja zmienia treść istniejącej notatki, czy odrzuca aktualizację pustą treścią, czy odrzuca aktualizację nieznanej notatki, czy usuwa notatkę i czy listuje tylko notatki danego ucznia
 - ReportService - czy raport agreguje obecności z okresu, czy uwzględnia daty graniczne, czy odrzuca niepoprawny zakres dat, czy eksport tworzy plik w żądanym formacie i czy odrzuca niepoprawne argumenty eksportu
- lessons
 - Lesson - czy konstruktor zapisuje pola, czy domyślny status to SCHEDULED, czy cena końcowa uwzględnia rabat, czy liczba wolnych miejsc liczy aktywne rezerwacje, czy lista zapisanych pochodzi z aktywnych rezerwacji, czy materiały da się dodać i usunąć, czy usunięcie nieobecnego materiału zwraca false, czy można dodać obecności i czy lista materiałów jest read-only
 - Reservation - czy konstruktor łączy ucznia z lekcją i ustawia datę, czy nowa rezerwacja ma status PENDING i jest aktywna, czy potwierdzenie ustawia CONFIRMED i czy anulowanie ustawia CANCELLED oraz dezaktywuje
 - Room - czy konstruktor zapisuje pola (nazwa, pojemność, wyposażenie) i czy settery aktualizują pola
 - LessonStatus - czy enum zawiera SCHEDULED, CANCELLED, COMPLETED
 - ReservationStatus - czy enum zawiera PENDING, CONFIRMED, CANCELLED

- LessonService - czy aktualizacja zmienia przedmiot i poziom trudności, czy odrzuca null lekcję, czy ustawienie ceny wpływa na cenę końcową, czy odrzuca ujemną cenę, czy odrzuca rabat spoza zakresu 0-100 i czy odrzuca argumenty null
- ReservationService - czy zwraca zaplanowane zajęcia z wolnymi miejscami, czy rezerwacja powodzi się i wysyła powiadomienie, czy odrzuca rezerwację przy braku miejsc, czy anulowanie respektuje regułę 24h, czy rezerwacja na okres tworzy wpis dla każdej lekcji, czy odrzuca niepoprawny zakres dat, czy eksport tworzy plik .ics, czy filtruje zajęcia po maksymalnej cenie (budżet) i czy zwraca null przy braku rezerwacji
- ScheduleService - czy zwraca wszystkie zajęcia, czy filtruje zajęcia po nauczycielu, czy przypisuje wolną salę, czy odrzuca salę zajęętą w tym czasie, czy lista wolnych sal pomija zajęte, czy zastępstwo zmienia prowadzącego, czy przełożenie zmienia terminy, czy anulowanie ustawia status CANCELLED i powiadamia uczestników oraz czy zwraca wolnych nauczycieli z pasującą dostępnością
- notifications
 - Notification - czy pełny konstruktor zapisuje wszystkie pola (z adresatem), czy skrócony konstruktor ustawia datę wysłania i czy settery aktualizują wartości
 - NotificationType - czy enum zawiera kanały SMS i EMAIL
 - NotificationService - czy potwierdzenie rezerwacji dociera do ucznia SMS-em, czy przypomnienie idzie SMS-em gdy włączone, czy przypomnienie jest wstrzymane gdy wyłączone, czy odwołanie powiadamia nauczyciela i zapisanych uczniów, czy powiadomienie o materiale dociera do zapisanych uczniów, czy ostrzeżenie o nieobecnościach dociera do ucznia i czy argumenty null są bezpiecznie ignorowane
- repository
 - InMemoryRepository - czy zapis dodaje encję a findAll ją zwraca, czy ignoruje null i duplikaty, czy usuwa encję, czy findAll zwraca kopię read-only i czy clear czyści repozytorium
 - LessonRepository - czy zwraca tylko zajęcia danego nauczyciela, czy pomija zajęcia odwołane, czy filtruje po zakresie dat i czy zwraca zajęcia, na które uczeń ma rezerwację
 - ReservationRepository - czy zwraca rezerwacje danego ucznia i czy zwraca rezerwacje danej lekcji
 - UserRepository - czy pracownicy nie zawierają uczniów i czy zwraca tylko nauczycieli
 - AttendanceRepository - czy zwraca wpisy danego ucznia, czy zwraca wpisy danej lekcji i czy filtruje wpisy po zakresie dat
- users
 - User - czy pełne imię łączy imię i nazwisko, czy radzi sobie z brakiem nazwiska, czy można przypisać dane kontaktowe, czy powiadomienia są przechowywane i read-only, czy dodanie null jest ignorowane i czy konto jest domyślnie aktywne oraz da się je dezaktywować

- Student - czy przypomnienia SMS są domyślnie wyłączone i przełączalne, czy można ustawić telefon rodzica, czy dziedziczy pełne imię z User, czy rezerwacje są zbierane i read-only oraz czy zbierane są obecności i notatki
- Teacher - czy zbierane są prowadzone zajęcia, czy ustawienie dostępności zastępuje poprzednią, czy zbierane są utworzone notatki i czy lista dostępności jest read-only
- Administrator - czy może edytować szczegóły zajęć, czy może edytować obecność i czy jest typu User (dziedziczenie)
- Manager - czy zbierane są wygenerowane raporty, czy lista raportów jest read-only i czy dziedziczy pełne imię
- ContactInfo - czy konstruktor zapisuje pola (e-mail, telefon) i czy settery aktualizują pola
- TeacherAvailability - czy przedział jest poprawny gdy początek przed końcem, czy niepoprawny gdy początek po końcu, czy wykrywa nakładanie w tym samym dniu, czy różne dni się nie nakładają i czy sąsiadujące przedziały się nie nakładają
- AvailabilityService - czy przyjmuje niekolidujące terminy, czy odrzuca nakładające się terminy, czy odrzuca niepoprawny przedział, czy odrzuca argumenty null, czy submit rzuca wyjątek przy konflikcie, czy submit przechodzi dla poprawnych terminów, czy zwraca terminy danego nauczyciela i czy agreguje dostępność wszystkich nauczycieli
- UserService - czy domyślnie tworzy nauczyciela, czy tworzy pracownika zgodnie z rolą, czy lista pracowników pomija uczniów, czy dezaktywacja ustawia konto jako nieaktywne, czy aktualizacja zapisuje użytkownika i czy odrzuca argumenty null

1.2.2 Testy akceptacyjne

- acceptance
 - ReservationAcceptanceTest - czy uczeń rezerwuje zajęcia i dostaje potwierdzenie SMS, czy rezerwacja jest odrzucana przy braku miejsc, czy można anulować ponad 24h przed zajęciami i czy anulowanie poniżej 24h jest odrzucane (brak zwrotu środków)
 - AttendanceAcceptanceTest - czy przekroczenie progu nieobecności powiadamia ucznia, czy liczba na progu nie wyzwała powiadomienia i czy lista obecności zawiera zapisanych uczniów
 - MaterialsAcceptanceTest - czy pełny cykl materiału (dodanie, powiadomienie, pobranie, usunięcie) działa i czy materiał ponad 100 MB jest odrzucany
 - ProgressReportAcceptanceTest - czy pełny cykl notatek (utworzenie, edycja, usunięcie, historia) działa i czy pusta notatka jest odrzucana
 - ReportingAcceptanceTest - czy manager generuje i eksportuje raport obecności i czy raport pomija obecności spoza zakresu
 - AvailabilityAcceptanceTest - czy nauczyciel zgłasza niekolidujące terminy, czy kolidujące terminy są odrzucane, czy usunięcie terminu z aktywną rezerwacją

jest odrzucane i czy dostępność obejmująca zarezerwowane zajęcia jest przyjmowana

2 Implementacja UI

Zaimplementowano prototyp UI w wersji high-fidelity. Całość znajduje się w katalogu UI/ na githubie. Zawiera 3 sekcje:

- Admin
- Nauczyciel
- Rodzic